Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №1

Вариант В32

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-34Б _____ (Ковалев М.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

Цель работы: разработать программу на языке Python для предметной области «Колонка данных - Таблица данных» и выполнить запросы варианта В. Закрепить навыки работы с классами, списками объектов и связями между ними.

Задачи:

1. Создать классы для хранения данных о колонках и таблицах.
2. Реализовать связи один-ко-многим и многие-ко-многим.
3. Подготовить тестовые данные с согласованными идентификаторами.
4. Выполнить три запроса:
   - вывести колонки, названия которых начинаются с буквы «А», и названия их таблиц;
   - вывести таблицы с минимальным размером значения колонки в каждой из них, отсортировав результат по возрастанию этого размера;
   - вывести все пары «колонка - таблица» для связи многие-ко-многим, отсортировав их по названиям колонок.
5. Проверить результаты выполнения программы.

Теоретическая часть

Класс в Python описывает структуру объектов и объединяет связанные данные. В работе используются классы `DataColumn`, `DataTable` и `ColumnTable`. Декоратор `@dataclass` автоматически создаёт метод инициализации объектов по объявленным полям.

При связи один-ко-многим одной таблице соответствует несколько колонок, а каждая колонка относится к одной таблице. Эта связь задаётся полем `table_id` в классе `DataColumn`.

Связь многие-ко-многим позволяет связать одну колонку с несколькими таблицами, а одну таблицу - с несколькими колонками. Для её представления используется класс `ColumnTable`, содержащий идентификаторы колонки и таблицы.

Обработка данных выполняется с помощью списковых и словарных включений. Метод `startswith()` используется для отбора названий по первой букве, функция `min()` - для определения минимального размера значения, а `sorted()` совместно с `itemgetter()` - для сортировки результатов по выбранному полю.

Код программы:

```
from dataclasses import dataclass
from operator import itemgetter

@dataclass
class DataColumn:
    id: int
    name: str
    size: int
    table_id: int

@dataclass
class DataTable:
    id: int
    name: str

@dataclass
class ColumnTable:
    column_id: int
    table_id: int

tables = [
    DataTable(1, "Клиенты"),
    DataTable(2, "Товары"),
    DataTable(3, "Заказы"),
]

columns = [
    DataColumn(1, "Адрес", 120, 1),
    DataColumn(2, "Артикул", 20, 2),
    DataColumn(3, "Наименование", 80, 2),
    DataColumn(4, "Количество", 4, 3),
]

column_tables = [
    ColumnTable(1, 1),
    ColumnTable(2, 2),
    ColumnTable(3, 2),
    ColumnTable(4, 3),
    ColumnTable(1, 3),
]

def main():
    one_to_many = [
```

```
    (column.name, column.size, table.name)
    for column in columns
    for table in tables
    if column.table_id == table.id
]

result_1 = [
    (name, table_name)
    for name, _, table_name in one_to_many
    if name.startswith("A")
]

sizes_by_table = {
    table.id: [
        column.size for column in columns
        if column.table_id == table.id
    ]
    for table in tables
}
result_2 = sorted(
    [
        (table.name, min(sizes_by_table[table.id]))
        for table in tables if sizes_by_table[table.id]
    ],
    key=itemgetter(1),
)

many_to_many = [
    (column.name, table.name)
    for link in column_tables
    for column in columns
    for table in tables
    if link.column_id == column.id
    and link.table_id == table.id
]
result_3 = sorted(many_to_many, key=itemgetter(0))

for number, result in enumerate(
    (result_1, result_2, result_3), start=1
):
    print(f"Задание B{number}")
    for name, value in result:
        print(f"{name}: {value}")
    print()
```

```
if __name__ == "__main__":
    main()
```

Вывод: в ходе работы разработана программа для предметной области «Колонка данных - Таблица данных». Реализованы обе связи и три запроса варианта В. Программа отобрала колонки «Адрес» и «Артикул», упорядочила таблицы по минимальным размерам значений 4, 20 и 120 байт и вывела связанные пары по названиям колонок. Результаты выполнения соответствуют тестовым данным.